INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 5: Convolutional Neural Networks

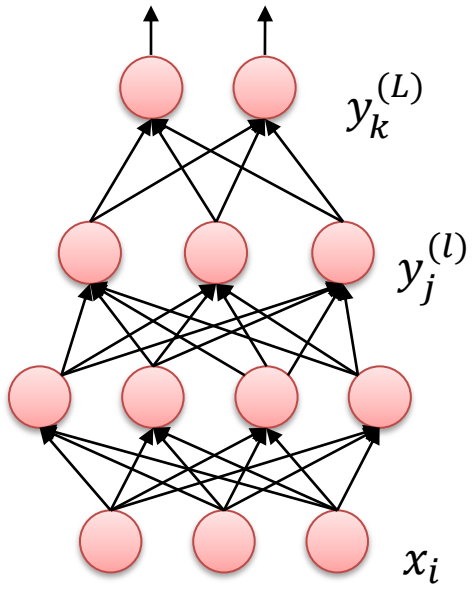
Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

Tsinghua University

Last lecture review

I. Multi-layer perceptron



- Forward calculation
- Backward calculation

II. Layer decomposition

- FC layer
- ReLU layer
- sigmoid layer
- tanh layer
- Euclidian loss layer
- Softmax CE loss layer

III. Training techniques-I

- Weight initialization
- Learning rate
- Momentum
- Randomly shuffle

Backward calculation (Exercise)

- Derive the local sensitivity δ and gradient $\partial E/\partial w$ and $\partial E/\partial b$ where applicable for
  - Euclidean loss layer: $E^{(n)} = \frac{1}{2}\left\|\boldsymbol{y}^{(L)} - \boldsymbol{t}\right\|^2$ ← $L+1$ layers in total
    - Note that here we calculate $\boldsymbol{\delta}^{(L)} = \partial E^{(n)}/\partial \boldsymbol{y}^{(L)}$
  - Softmax-cross-entropy error layer $E^{(n)} = -\sum_{k=1}^{K} t_k \ln f\left(y_k^{(L)}\right)$ ← L layers in total
    - Note that here we calculate $\boldsymbol{\delta}^{(L-1)} = \partial E^{(n)}/\partial \boldsymbol{y}^{(L-1)}$
  - Fully connected layer: $y_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$
  - Sigmoid layer: $y_j^{(l)} = f(y_j^{(l-1)})$, where f is a sigmoid function
  - ReLU layer: $y_j^{(l)} = f(y_j^{(l-1)})$, where f is a ReLU function

Note: $\boldsymbol{u}^{(l)}$ and $\boldsymbol{y}^{(l)}$ are identical in every layer l

Answers

$\boldsymbol{u}^{(l)}$ and $\boldsymbol{y}^{(l)}$ are identical in every layer l

1. Euclidean loss layer $\quad \boldsymbol{\delta}^{(L)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$
2. Softmax-cross-entropy error layer $\quad \boldsymbol{\delta}^{(L-1)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$

   must be softmax output
3. The l-th layer is an FC layer

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\boldsymbol{y}^{(l-1)})^\top, \quad \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}, \quad \boldsymbol{\delta}^{(l-1)} = (\boldsymbol{W}^{(l)})^\top \boldsymbol{\delta}^{(l)}$$

4. The l-th layer is a sigmoid layer

$$\boldsymbol{\delta}^{(l-1)} = \boldsymbol{\delta}^{(l)} \odot \boldsymbol{f}'(\boldsymbol{y}^{(l-1)}) \text{ where } \quad f'(x) = f(x)(1 - f(x))$$

5. The l-th layer is a relu layer

$$\boldsymbol{\delta}^{(l-1)} = \boldsymbol{\delta}^{(l)} \odot \boldsymbol{f}'(\boldsymbol{y}^{(l-1)}) \text{ where } \quad f'(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{else.} \end{cases}$$

BP in vector-matrix form

- The last layer
  - For MSE layer, calculate $\boldsymbol{\delta}^{(L)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$
  - For Softmax-CE layer, calculate $\boldsymbol{\delta}^{(L-1)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$
- From the last layer to the first layer
  - The l-th layer is a FC layer

    $$\frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\boldsymbol{y}^{(l-1)})^\top, \quad \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}, \quad \boldsymbol{\delta}^{(l-1)} = (\boldsymbol{W}^{(l)})^\top \boldsymbol{\delta}^{(l)}$$

  - The l-th layer is an activation layer

    $\boldsymbol{\delta}^{(l-1)} = \boldsymbol{\delta}^{(l)} \odot \boldsymbol{f}'(\boldsymbol{y}^{(l-1)})$ where $f(\cdot)$ is the act fun

for each sample n

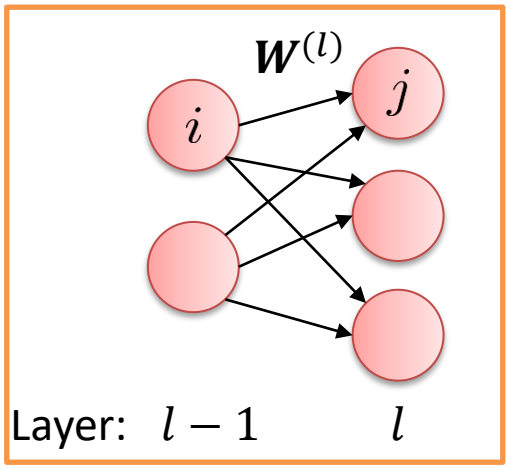
- Update weights

avg over n

$$\boldsymbol{W}^{(l)} = \boldsymbol{W}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} - \alpha\lambda \boldsymbol{W}^{(l)}, \quad \boldsymbol{b}^{(l)} = \boldsymbol{b}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}}$$

Paper reading

- Form groups of two and every group chooses one of the following papers for reading
- Share with us the main conclusion and your opinion on it.
  - Smith, Cyclical learning rates for training neural networks, IEEE Winter Conference on Applications of Computer Vision, 2017
  - Liao, Leibo, Poggio, How important is weight symmetry in backpropagation? AAAI, 2016



$$\boldsymbol{u}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)} + \boldsymbol{b}^{(l)}$$

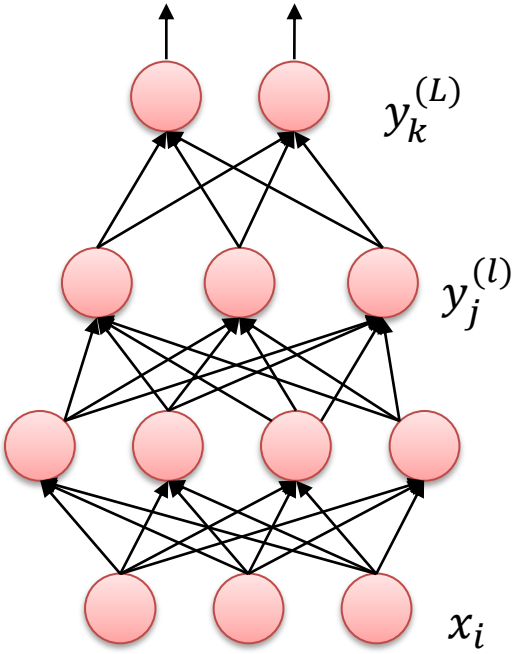
$$\boldsymbol{\delta}^{(l-1)} = \left(\boldsymbol{W}^{(l)}\right)^{\mathsf{T}}\boldsymbol{\delta}^{(l)}$$

Outline

I. Introduction
II. Convolutional layer
III. Pooling layer
IV. Summary

Motivation

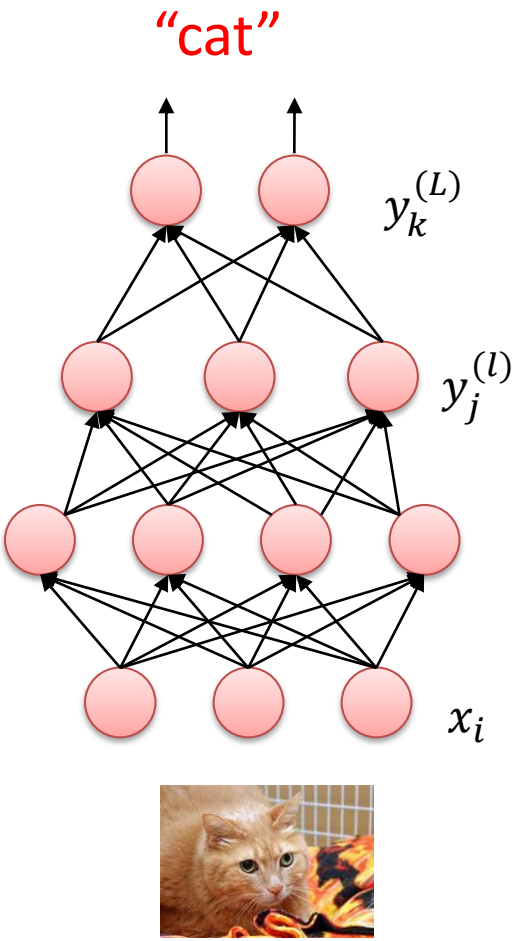
How to use MLP to classify images?



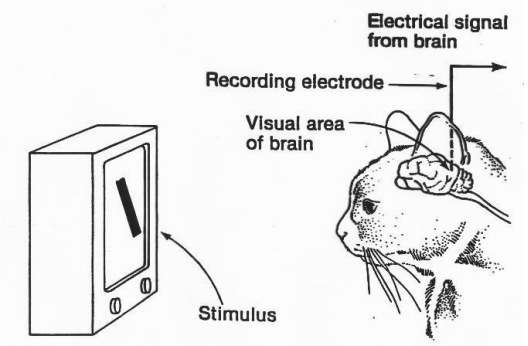
Motivation

How to use MLP to classify images?

Are there any shortcomings?



Hubel and Wiesel's experiment



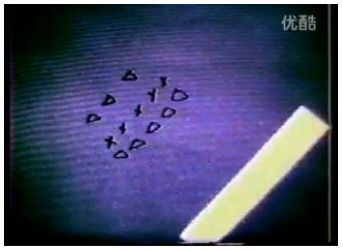


Novel Prize 1981

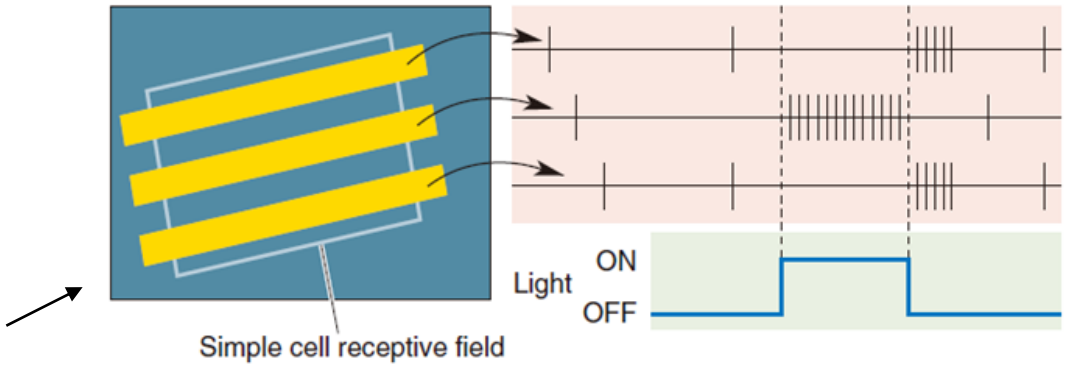


Youku link

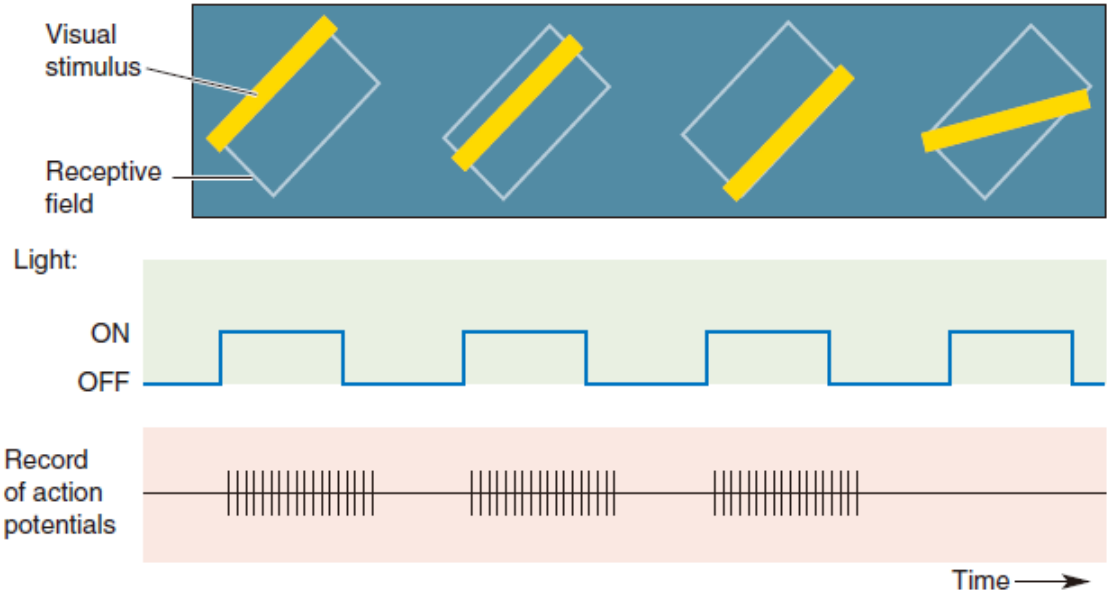
Simple cell and complex cell



Simple cells: local feature detectors

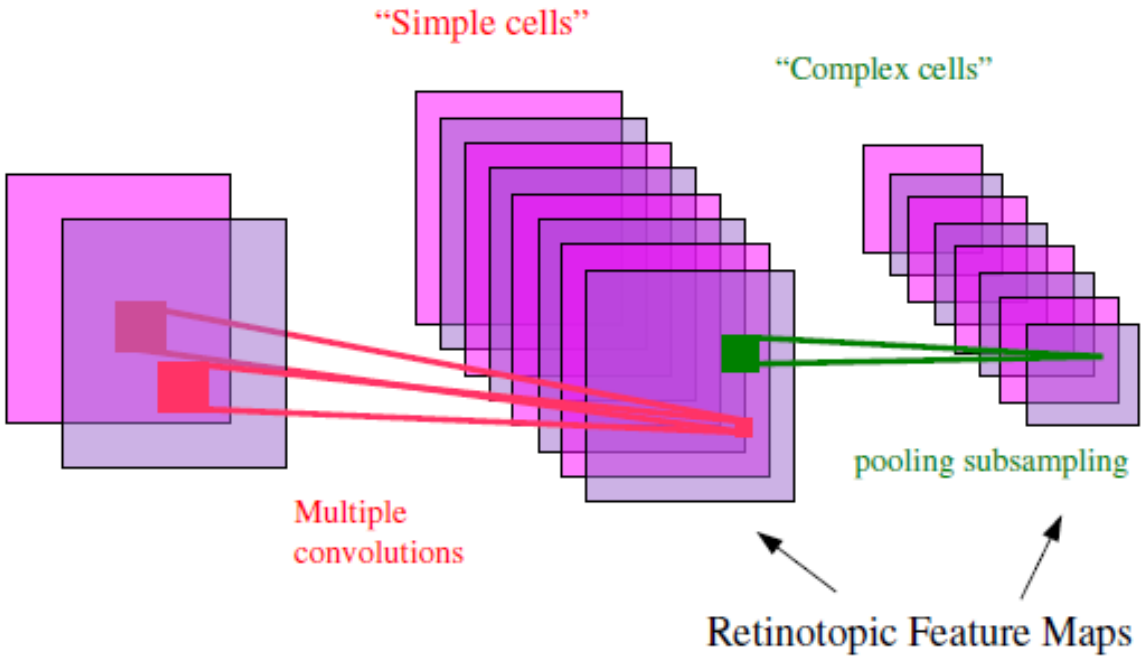


Complex cells: translation invariance!

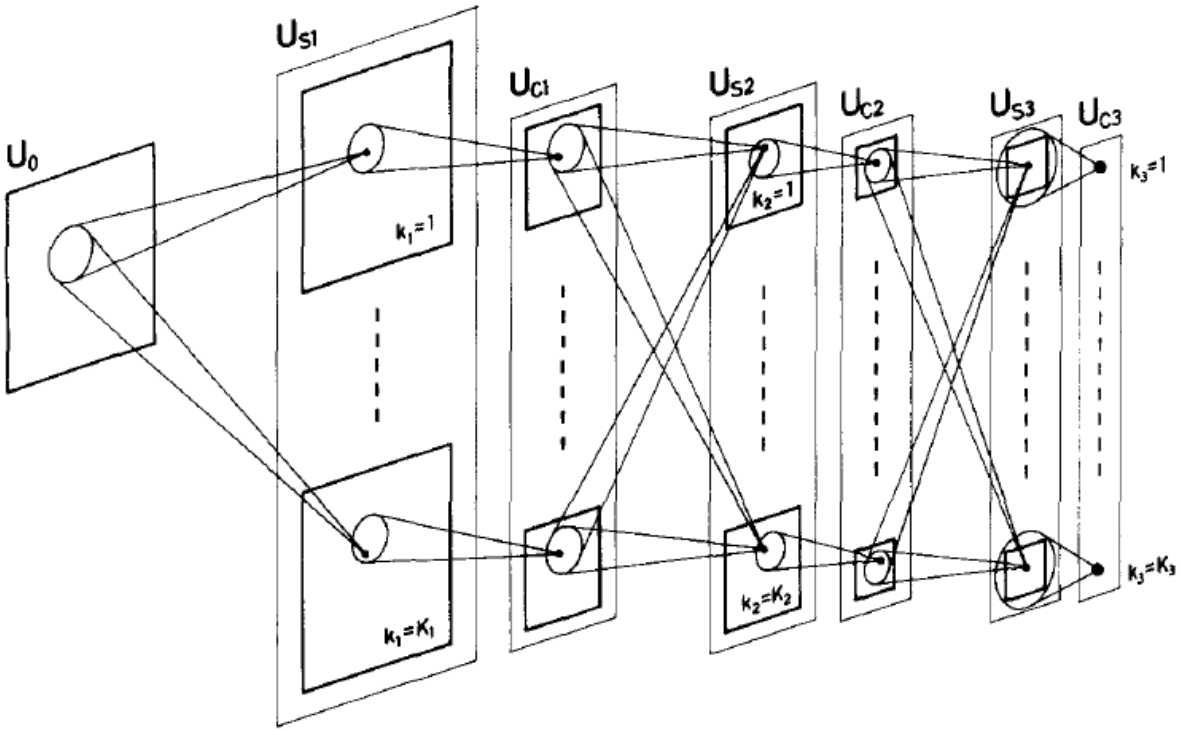


Local detectors and shift invariance in the cortex

- (Hubel & Wiesel 1962)
  - Simple cells detect local features
  - complex cells "pool" the outputs of simple cells within a retinotopic neighborhood



Neocognitron



福岛邦彦



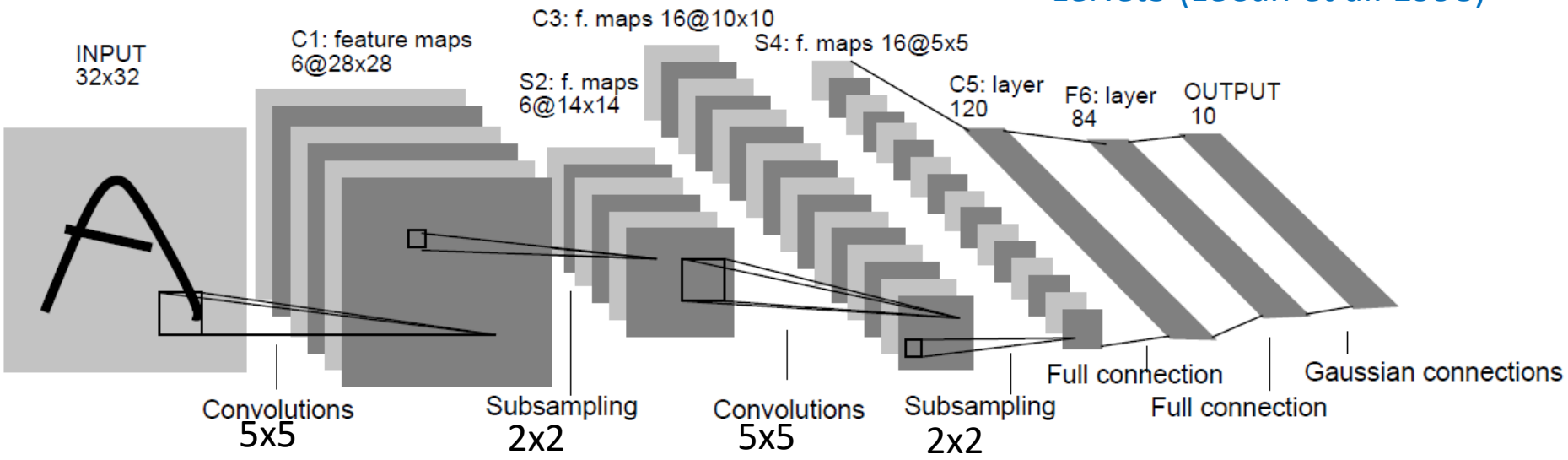
Kunihiko Fukushima, Biol. Cybernetics, 1980

Franklin Institute honors pioneers in the study of forest fires, longevity, eyesight

https://www.inquirer.com/science/franklin-institute-science-awards-climate-change-20200127.html

Convolutional neural network (CNN)

LeNet5 (LeCun et al. 1998)



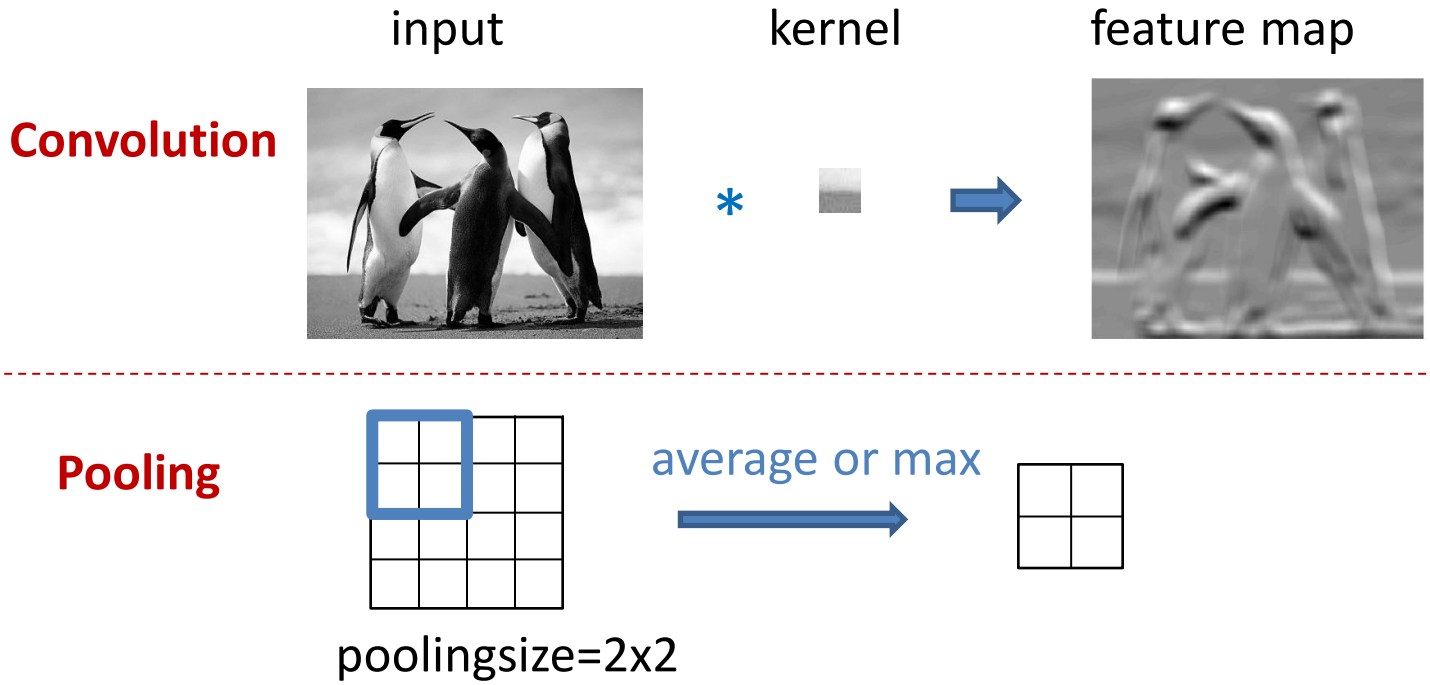
- Convolution
  - Local connections and weight sharing
- Subsampling (pooling)
- Contribution: apply BP algorithm

LeCun, B. Boser et al., Backpropagation applied to handwritten zip code recognition, *Neural Computation* (1989)



Yann LeCun (USA)
Turing award 2018

Two new layers



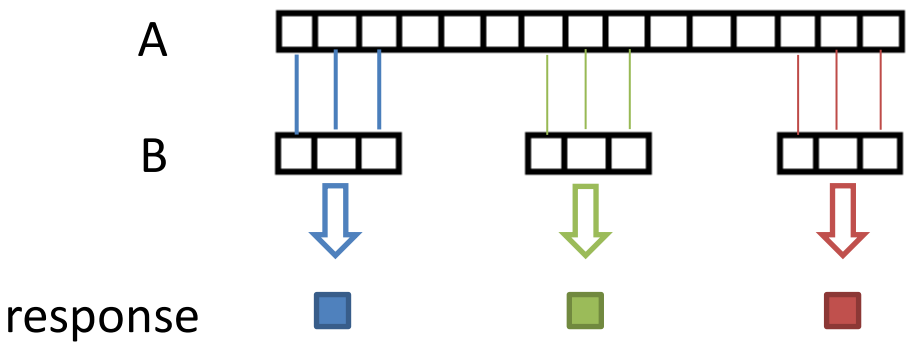
- Convolutional layer and pooling layer
  - Define two additional layers with forward computation and backward computation

Outline

I. Introduction
II. Convolutional layer
III. Pooling layer
IV. Summary

Motivation

- Suppose there are two 1D sequences A and B where the length of B is smaller than that of A
- Compute the similarity between B and each part of A
- Naively, we could slide B on A and calculate the similarity one by one
  - We call it "similarity calculation"



But this process could be slow. We use GPU

Cosine similarity between two vectors x and y:

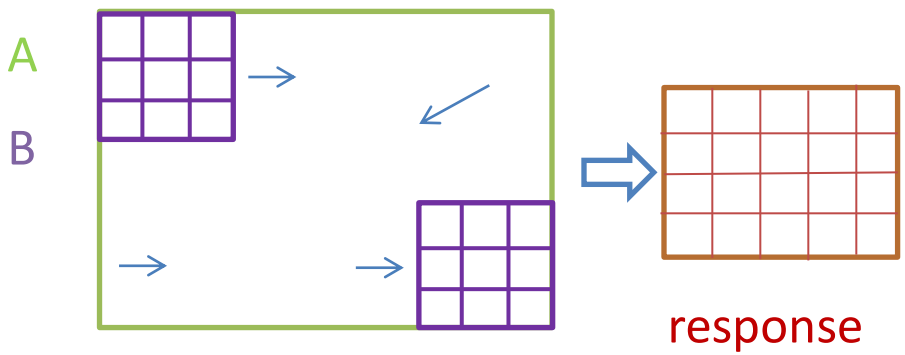
$$s \equiv \cos\theta = \frac{x^{\top} y}{||x||||y||}$$

$$= \sum_i x_i y_i$$

if the two vectors have unit length

Motivation

- Suppose there are two 2D images A and B where the size of B is smaller than that of A
- Compute the similarity between B and each part of A
- Naively, we could slide B on A and calculate the similarity one by one
  - We call it "similarity calculation"



Cosine similarity between two matrices x and y:

$$s = \sum_{i,j} x_{ij} y_{ij}$$

if the two matrices have unit Frobenius norm

But this process could be slow. We use GPU

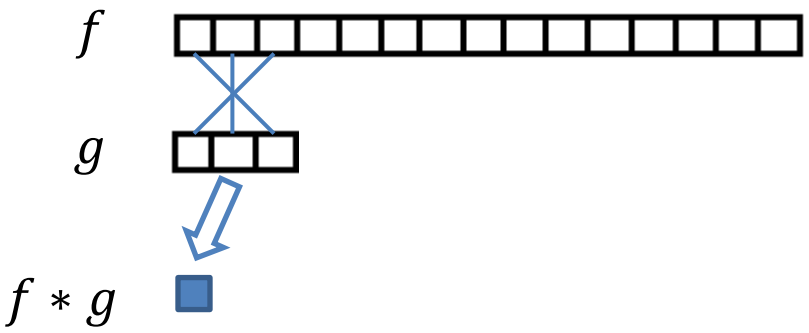
1D convolution

- Continuous convolution

$$(f * g)(t) \triangleq \int_{-\infty}^{\infty} f(\tau) g(t-\tau) d\tau = \int_{-\infty}^{\infty} f(t-\tau) g(\tau) d\tau$$

- Discrete convolution (for finite length sequences)

$$(f * g)[m] \triangleq \sum_{n=1}^{N} f[m-n] g[n]$$



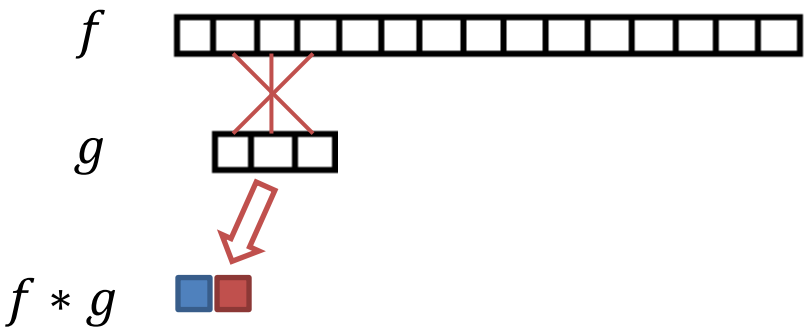
1D convolution

- Continuous convolution

$$(f * g)(t) \triangleq \int_{-\infty}^{\infty} f(\tau) g(t-\tau) d\tau = \int_{-\infty}^{\infty} f(t-\tau) g(\tau) d\tau$$

- Discrete convolution (for finite length sequences)

$$(f * g)[m] \triangleq \sum_{n=1}^{N} f[m-n] g[n]$$



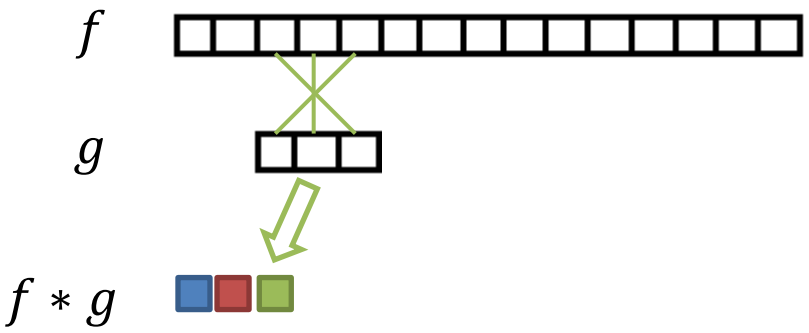
1D convolution

- Continuous convolution

$$(f * g)(t) \triangleq \int_{-\infty}^{\infty} f(\tau) g(t-\tau) d\tau = \int_{-\infty}^{\infty} f(t-\tau) g(\tau) d\tau$$

- Discrete convolution (for finite length sequences)

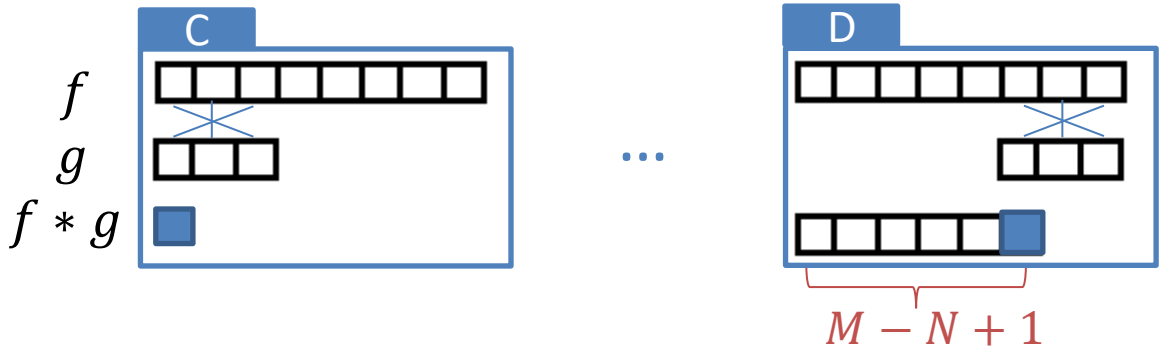
$$(f * g)[m] \triangleq \sum_{n=1}^{N} f[m-n] g[n]$$



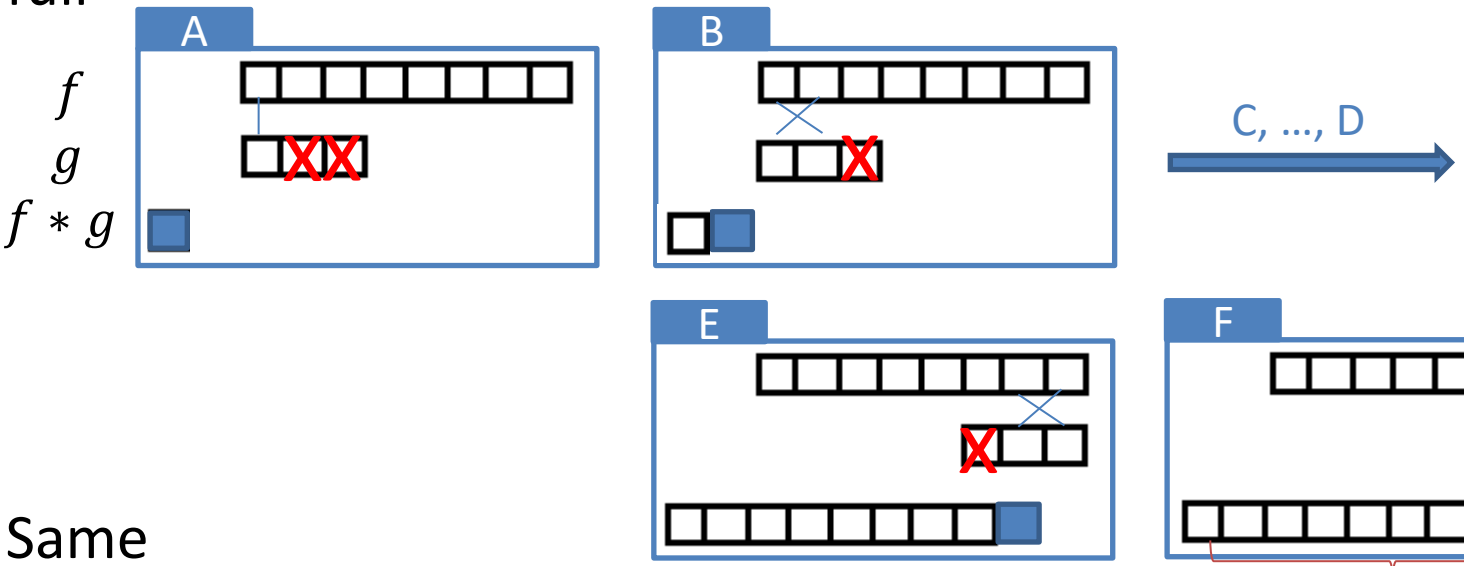
Three shapes of convolution

Length of f: M, length of g: N, where $M \geq N$

- valid



- full



- Same
  - truncate full result to M dimension

Example

- "Same" convolution can be also obtained by "valid" convolution of g with *zero-padded* f (we'll come back later)
- Suppose there are two sequences

$$f = [0, 1, 2, -1, 3]$$
$$g = [1, 1, 0]$$

- Then

$$(f * g)_{\text{valid}} = [3, 1, 2]$$
$$(f * g)_{\text{full}} = [0, 1, 3, 1, 2, 3, 0]$$
$$(f * g)_{\text{same}} = [1, 3, 1, 2, 3]$$

```
import numpy as np
from scipy import signal
```

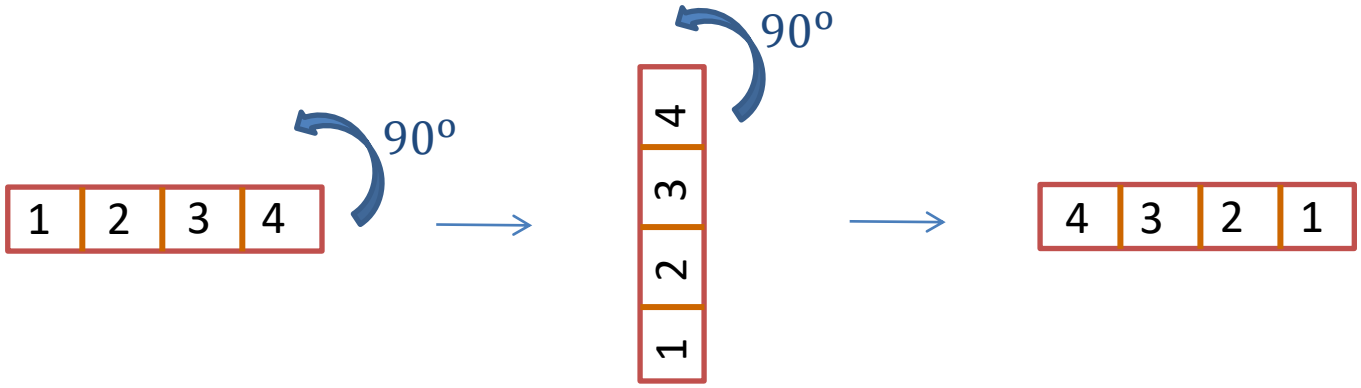
```
f = np.array([0,1,2,-1,3])
g = np.array([1,1,0])
h = signal.convolve(f,g,mode='valid')
h = signal.convolve(f,g,mode='full')
h = signal.convolve(f,g,mode='same')
```

Relationship between similarity and convolution

- Calculating the the similarity between sequence g and each part of sequence f is equivalent to calculating $f * \tilde{g}$ where

$$\tilde{g}_1 = g_N, \tilde{g}_2 = g_{N-1}, \cdots, \tilde{g}_N = g_1$$

- The above flip operation can be realized by applying the command numpy.rot90() twice (denoted by rot180() hereafter)

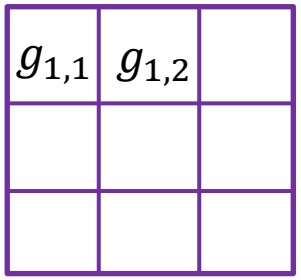


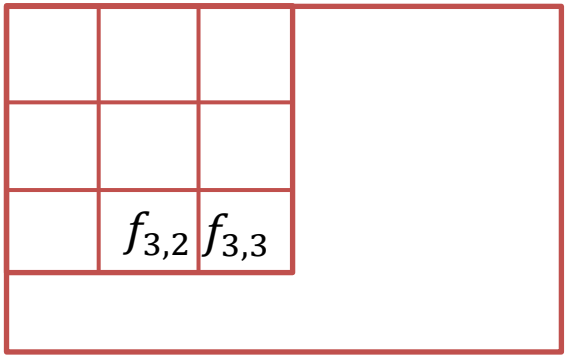
It's equivalent to *flip the vector along the axis 0*

2D convolution

- Suppose that there are two matrices f and g with sizes $M \times N$ and $K_1 \times K_2$, respectively, where $M \geq K_1, N \geq K_2$
- Discrete convolution of the two matrices

$$h[m,n] = (f * g)[m,n] \triangleq \sum_{k_1=1}^{K_1} \sum_{k_2=1}^{K_2} f[m-k_1, n-k_2]\, g[k_1, k_2]$$





When $m = 4, n = 4$

$$(f * g)_{m,n} = f_{3,3}g_{1,1} + f_{3,2}g_{1,2} + f_{3,1}g_{1,3} + f_{2,3}g_{2,1} + \cdots$$

- valid shape: the size of h is $(M - K_1 + 1) \times (N - K_2 + 1)$
- full shape: the size of h is $(M + K_1 - 1) \times (N + K_2 - 1)$
- same shape: the size of h is $M \times N$

Examples

```
import numpy
from scipy import signal
A = numpy.array([[0,0,1,2],[2,2,0,0],[2,1,2,2],[3,0,1,1]])
print(A)
```

0	0	1	2
2	2	0	0
2	1	2	2
3	0	1	1

```
B = numpy.array([[0,0,-1],[1,-1,1],[-1,1,1]])
print(B)
```

0	0	-1
1	-1	1
-1	1	1

```
C = signal.convolve2d(A,B,mode='full')
print(C)
```

0	0	0	0	-1	-2
0	0	-1	-1	-1	2
2	0	-3	0	1	0
0	-1	4	3	-1	1
1	-2	5	1	4	3
-3	3	2	0	2	1

```
C = signal.convolve2d(A,B,mode='valid')
print(C)
```

-3	0
4	3

Examples

```
import numpy
from scipy import signal
A = numpy.array([[0,0,1,2],[2,2,0,0],[2,1,2,2],[3,0,1,1]])
print(A)
```

0	0	1	2
2	2	0	0
2	1	2	2
3	0	1	1

```
B = numpy.array([[0,0,-1],[1,-1,1],[-1,1,1]])
print(B)
```

0	0	-1
1	-1	1
-1	1	1

```
C = signal.convolve2d(A,B,mode='full')
print(C)
```

0	0	0	0	-1	-2
0	0	-1	-1	-1	2
2	0	-3	0	1	0
0	-1	4	3	-1	1
1	-2	5	1	4	3
-3	3	2	0	2	1

```
C = signal.convolve2d(A,B,mode='same')
print(C)
```

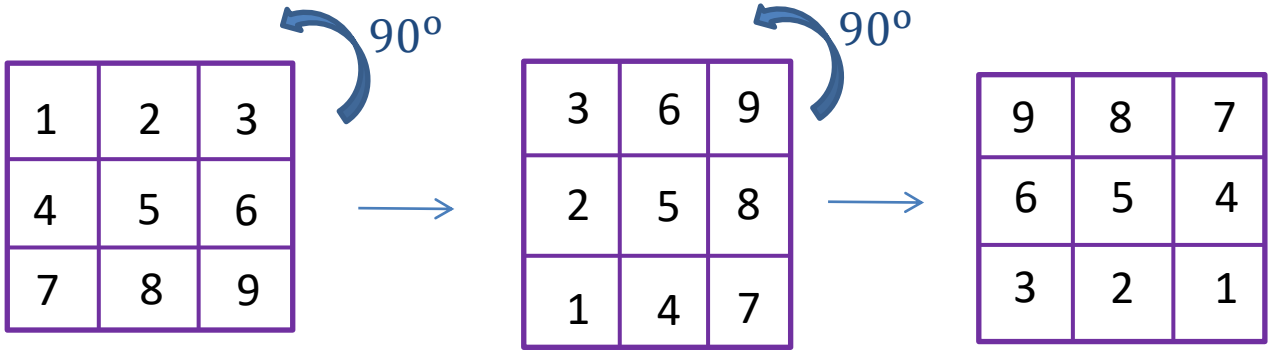
0	-1	-1	-1
0	-3	0	1
-1	4	3	-1
-2	5	1	4

Relationship between similarity and convolution

- Calculating the the similarity between matrix g and each part of matrix f is equivalent to calculating $f * \tilde{g}$ where

$$\tilde{g}_{1,1} = g_{M,N}, \tilde{g}_{1,2} = g_{M,N-1}, \cdots, \tilde{g}_{1,N} = g_{M,1}$$
$$\tilde{g}_{2,1} = g_{M-1,N}, \tilde{g}_{2,2} = g_{M-1,N-1}, \cdots, \tilde{g}_{2,N} = g_{M-1,1}$$
$$\vdots \qquad\qquad \vdots$$
$$\tilde{g}_{M,1} = g_{1,N}, \tilde{g}_{M,2} = g_{1,N-1}, \cdots, \tilde{g}_{M,N} = g_{1,1}$$

- The above operation can be realized by applying the command numpy.rot90() twice (denoted by rot180() hereafter)



It's equivalent to *flip the matrix along the axes 0 then 1*

Convolution for local feature detection

Image
(mean subtracted)



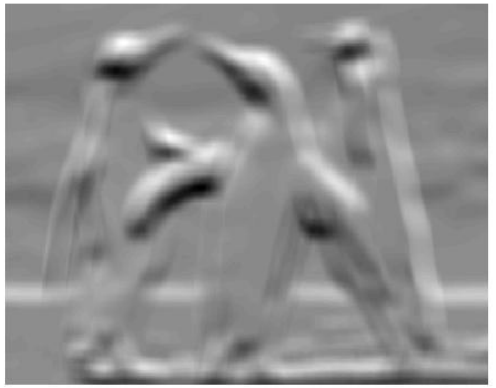
filter

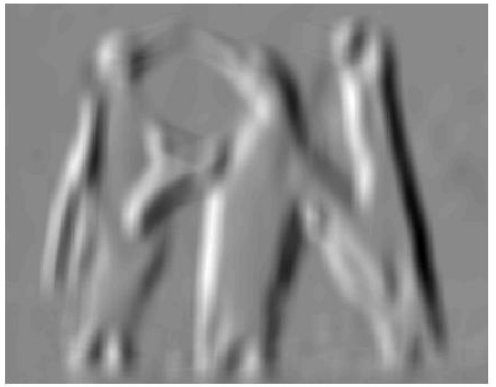


sim



feature map

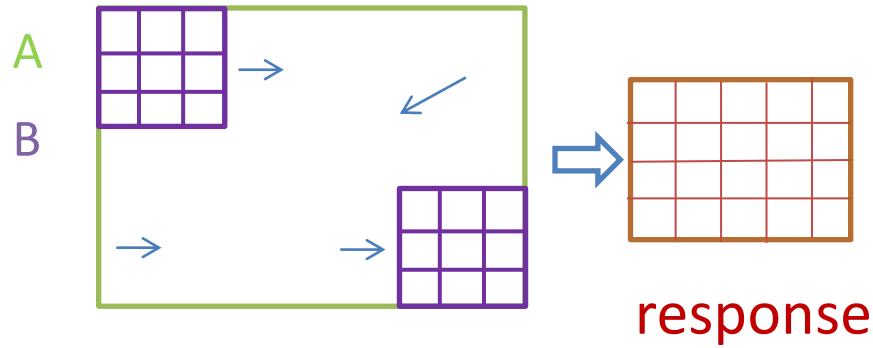




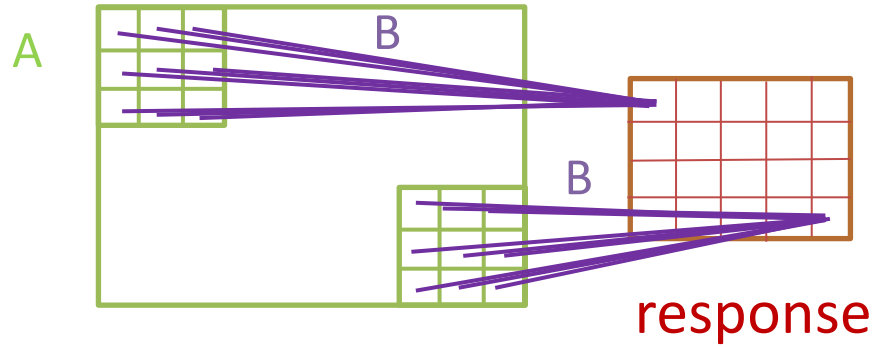
The higher a pixel value (brighter) in the feature map, the more similar between the filter and the corresponding patch in the figure

Interpretation from neural network perspective

- Compute the similarity between B and each part of A



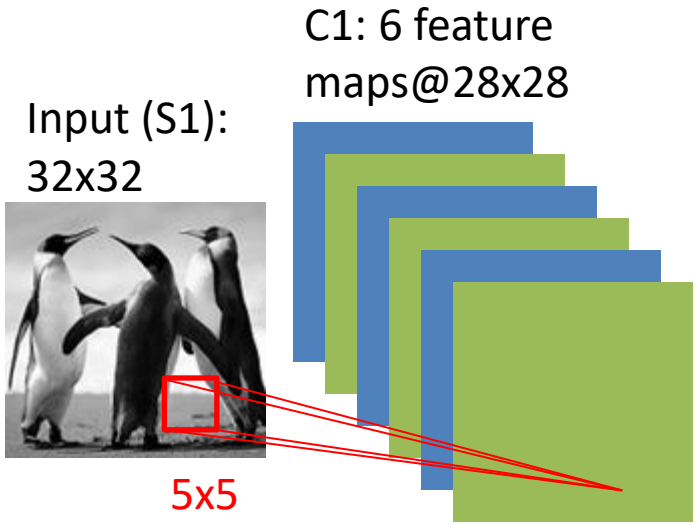
From the perspective of neural network

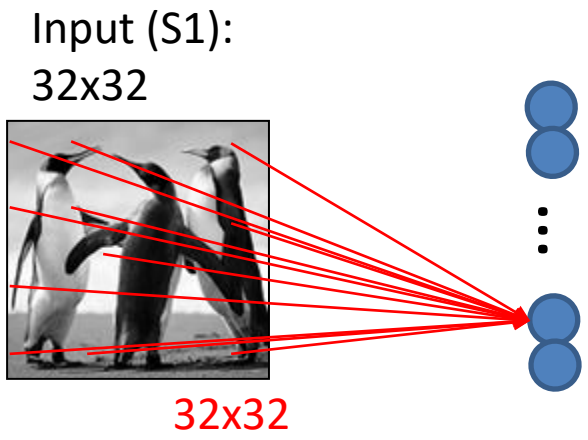


Local connections

weight sharing

Convolution saves the number of parameters





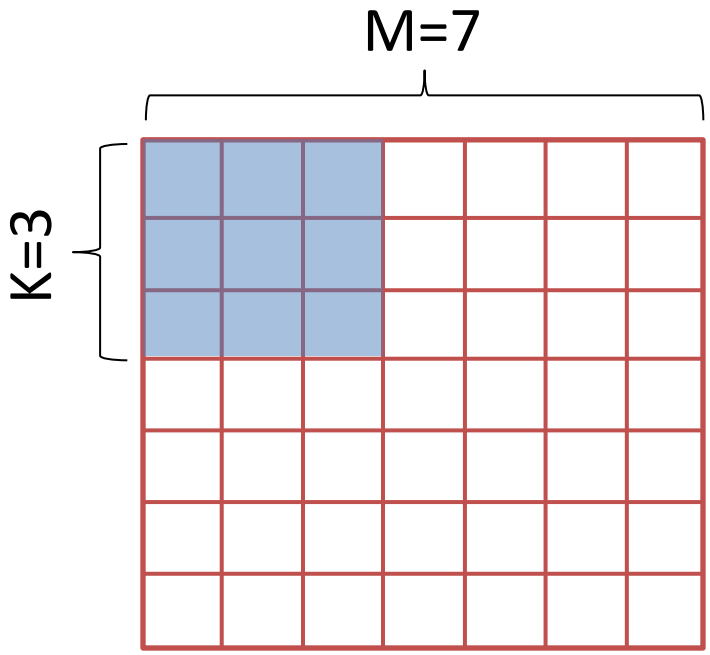
- One feature map has 25 parameters
- The total number of parameters:
  - 25x the number of feature maps

- One neuron has 1024 parameters
- The total number of parameters:
  - 1024x the number of neurons

Extensions

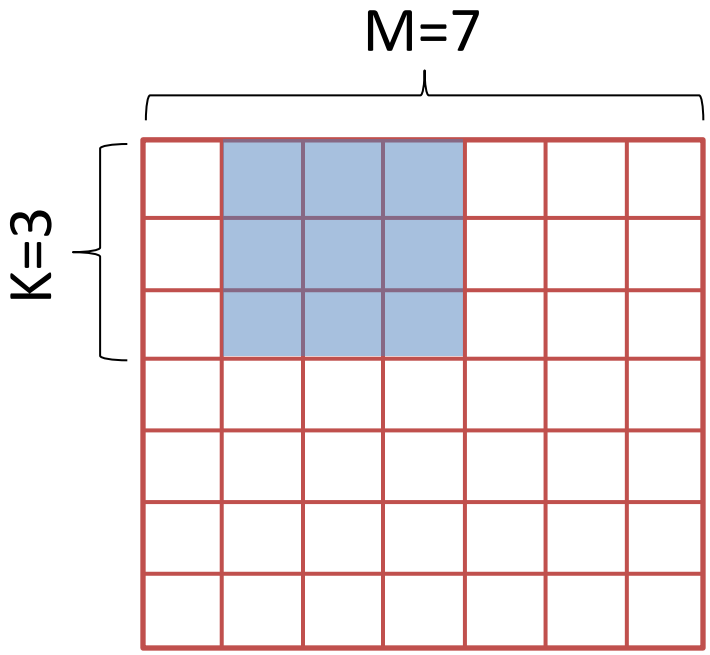
1. Preserving the spatial size with “valid” mode convolution
2. Convolution with stride≠1

1. Preserving the spatial size



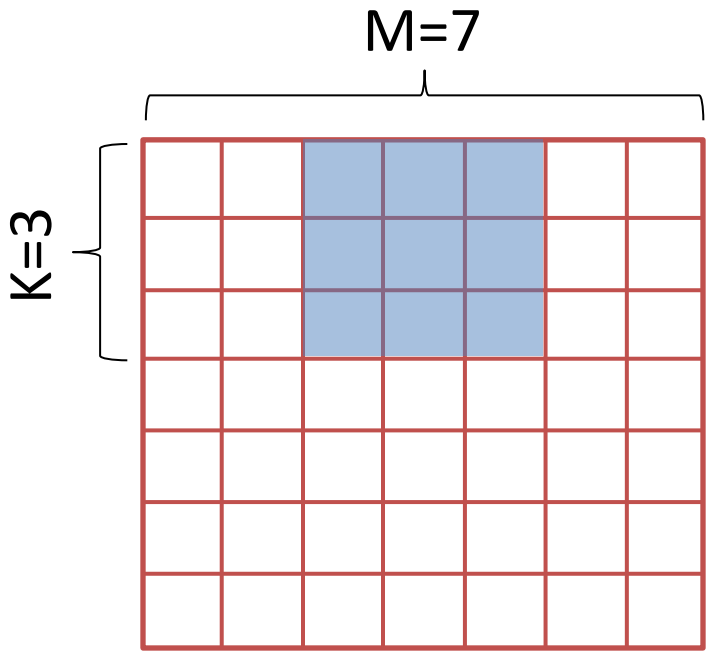
- Input size M=7x7
- Kernel size K=3x3
- Stride=1
- Output size (valid mode):

1. Preserving the spatial size



- Input size M=7x7
- Kernel size K=3x3
- Stride=1
- Output size (valid mode):

1. Preserving the spatial size



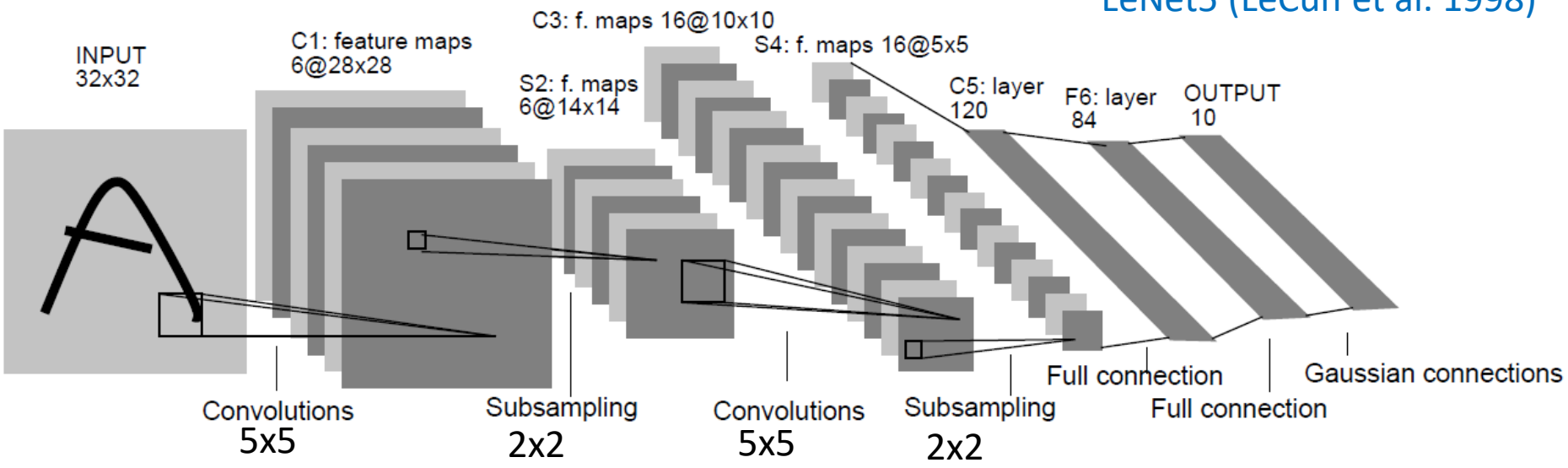
- Input size M=7x7
- Kernel size K=3x3
- Stride=1
- Output size (valid mode): 5x5

The input is shrunk

1. Preserving the spatial size

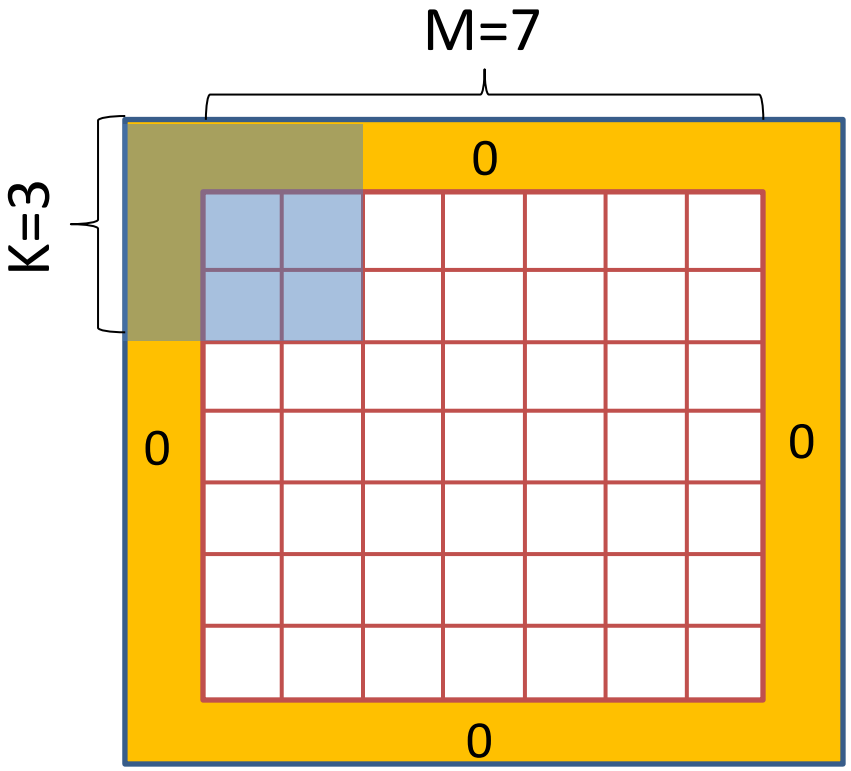
This is the case in LeNet 5

LeNet5 (LeCun et al. 1998)



If we don't want to shrink the input, what shall we do?

1. Preserving the spatial size



- Input size M=7x7
- Kernel size K=3x3
- Stride=1
- Pad with 1 pixel border
- Output size: 7x7

The "same" mode conv

- Usually, K is odd
- To keep the output size the same as input size, with stride=1, what is the pad size (on each side)?

单选题 1分

Setting

- Input size MxM. Kernel size KxK. Stride=1.
- If K is odd. To keep the output size the same as the input size, what is the pad size (on each side)?

A (M-K)/4

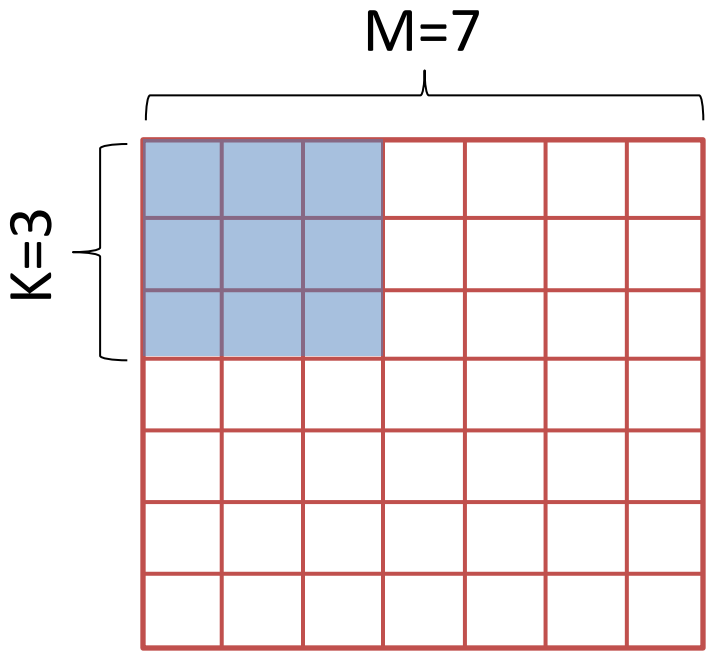
B (K+1)/2

C (K-1)/2

D (M-1)/2

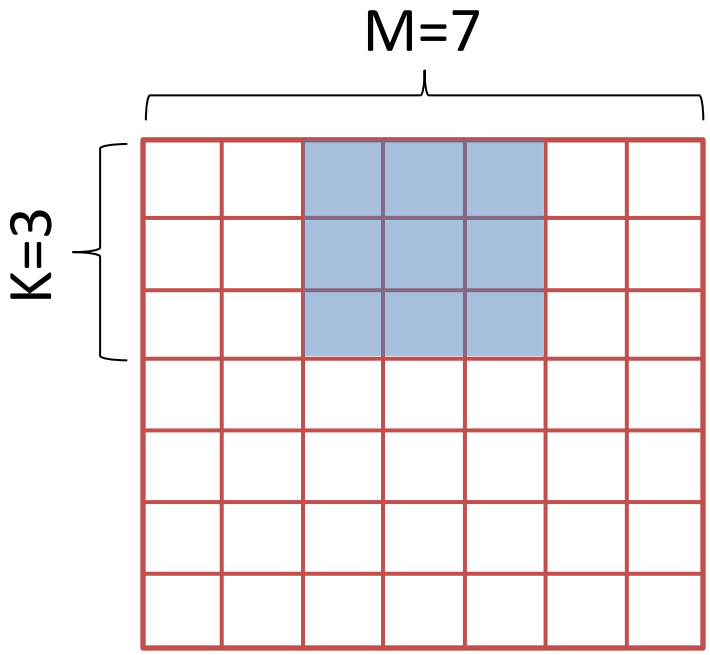
Submit

2. Convolution with stride≠1



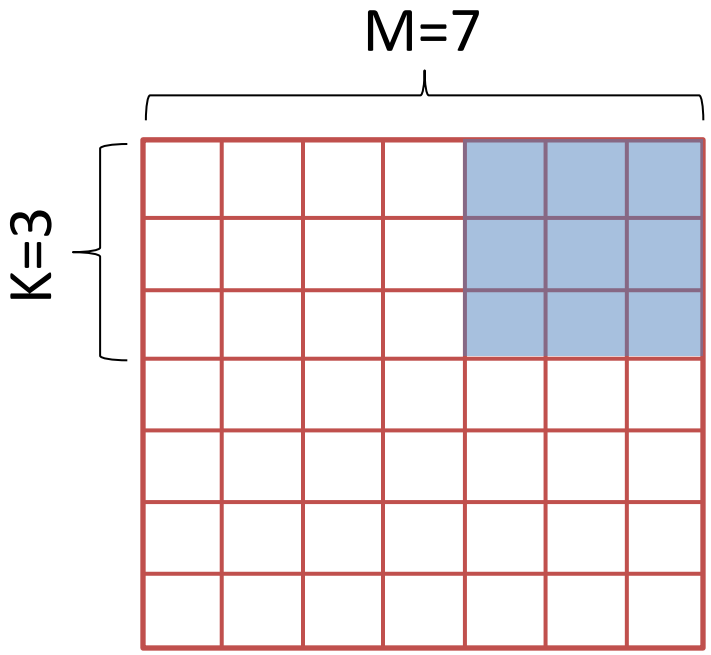
- Input size M=7x7
- Kernel size K=3x3
- Stride=2
- Output size (valid mode):

2. Convolution with stride≠1



- Input size M=7x7
- Kernel size K=3x3
- Stride=2
- Output size (valid mode):

2. Convolution with stride≠1



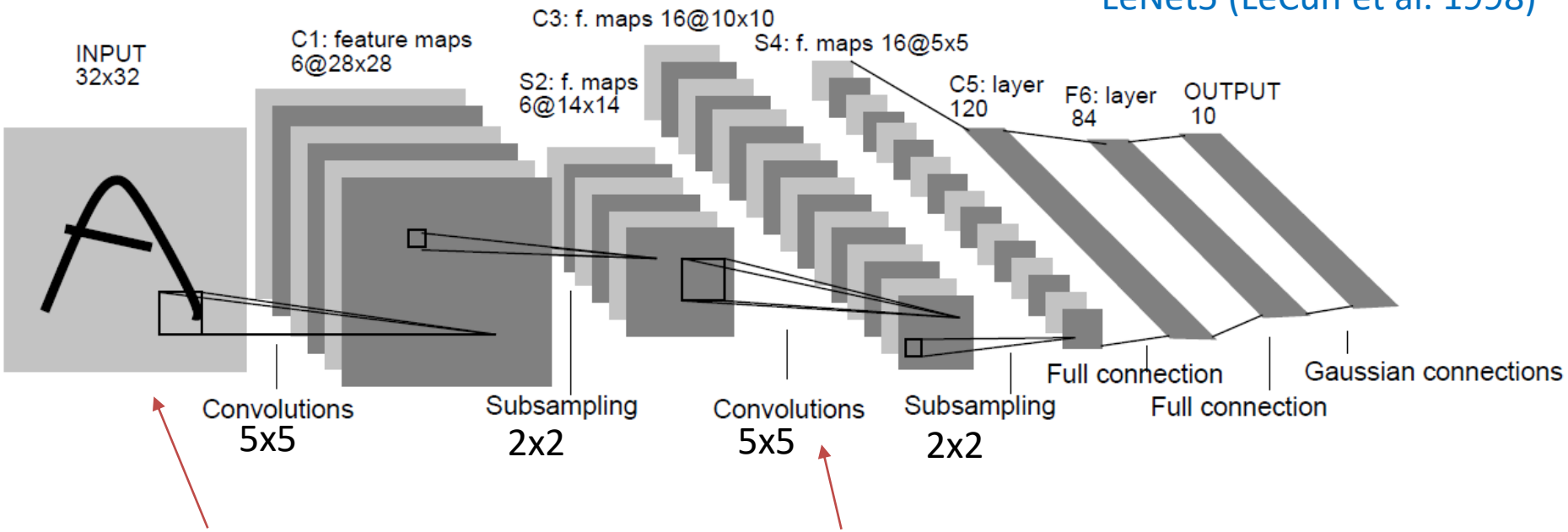
- Input size M=7x7
- Kernel size K=3x3
- Stride=2
- Output size (valid mode): 3x3

In general, output size: (M-K)/stride+1

What if (M-K)/stride is not an integer?

3D convolution

LeNet5 (LeCun et al. 1998)

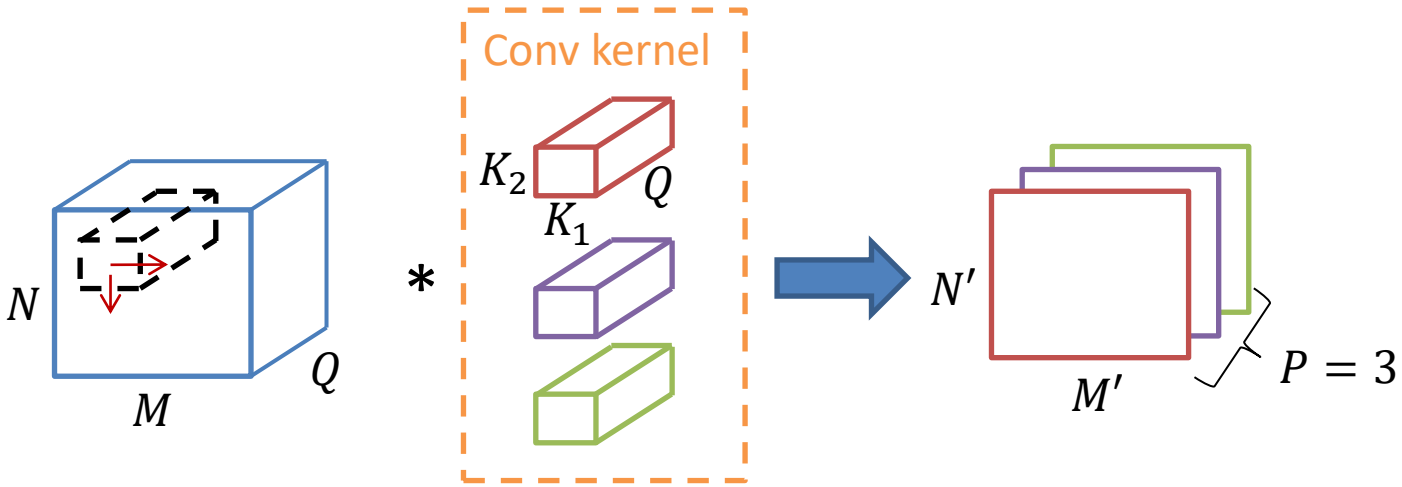


What if the input is an RGB image?

How to do this convolution?

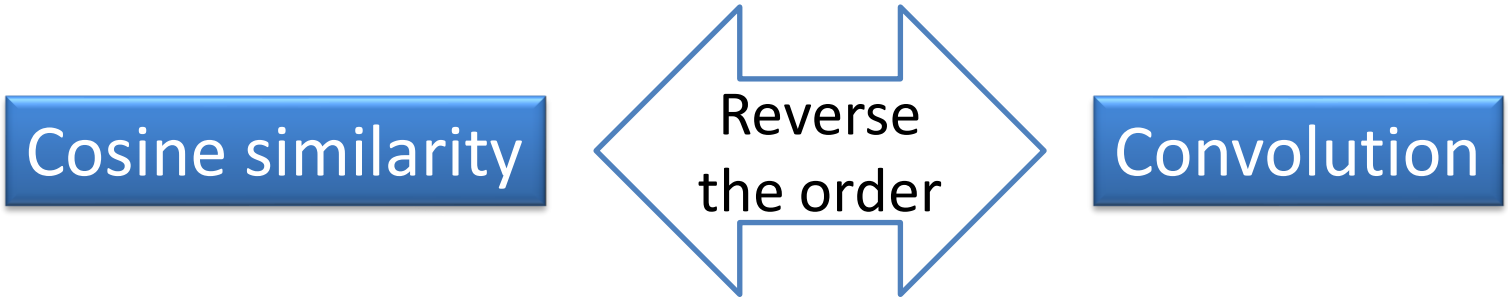
3D convolution

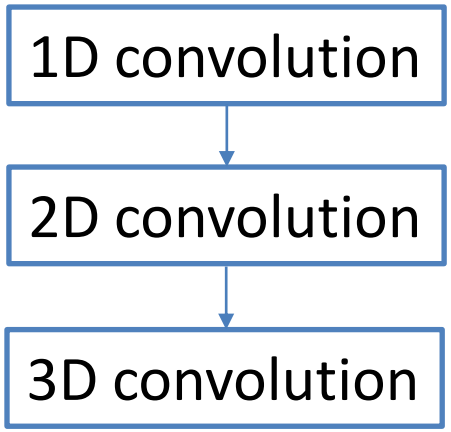
- We assume the number of channels in the input is the same as that in the kernel (filter)
- Convolve a 2D feature map in the 3D input with the *corresponding* 2D section in the 3D kernel, then sum over all sections to yield one feature map
  - This can be realized by flipping the 3D kernel and do 3D convolution



The number of parameters in this layer is $P \times Q \times K_1 \times K_2$

Summary of Part 2





stride

Padding

多选题 1分

Setting

What claims about a convolutional layer are correct ?

A Look for locally matched patterns

B Reduce the number of parameters

C Increase the number of parameters

D Simulate the functions of simple cells in the brain

Submit

单选题 1分

设置

Is the following claim correct?
3D convolution is used in a neural network only when the input image is a color image

A Yes

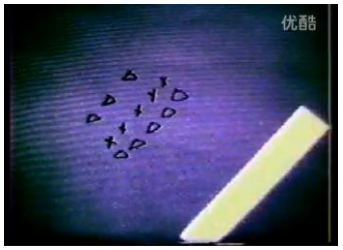
B No

提交

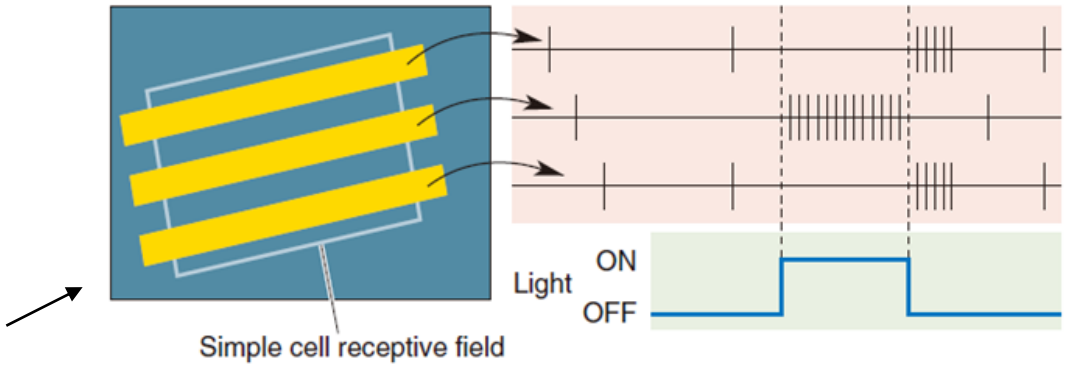
Outline

I. Introduction
II. Convolutional layer
III. Pooling layer
IV. Summary

Simple cell and complex cell



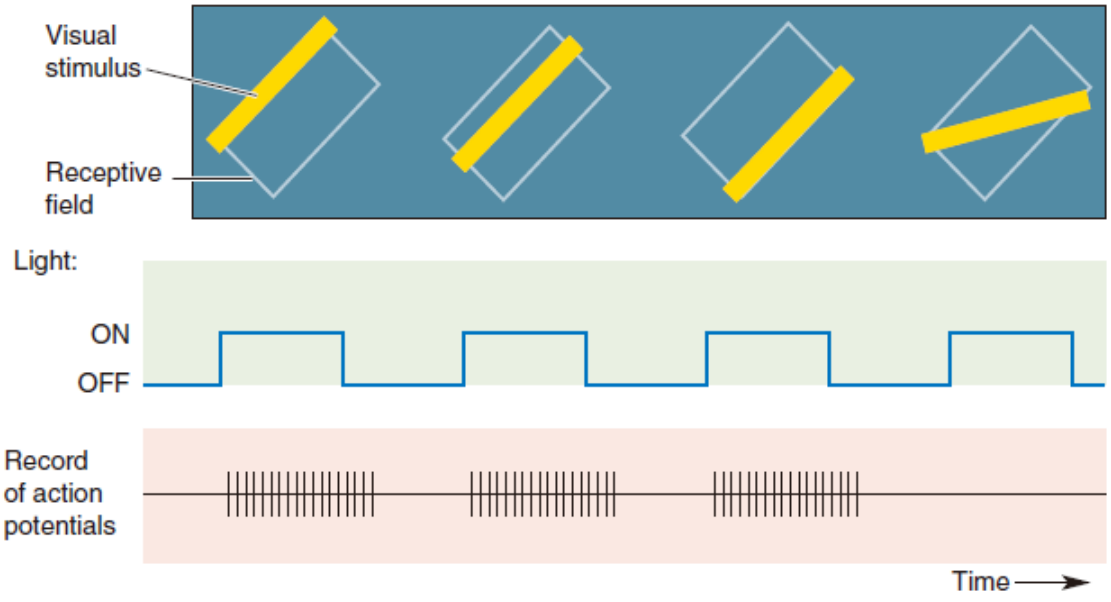
Simple cells: local feature detectors



What's the advantage of complex cells?

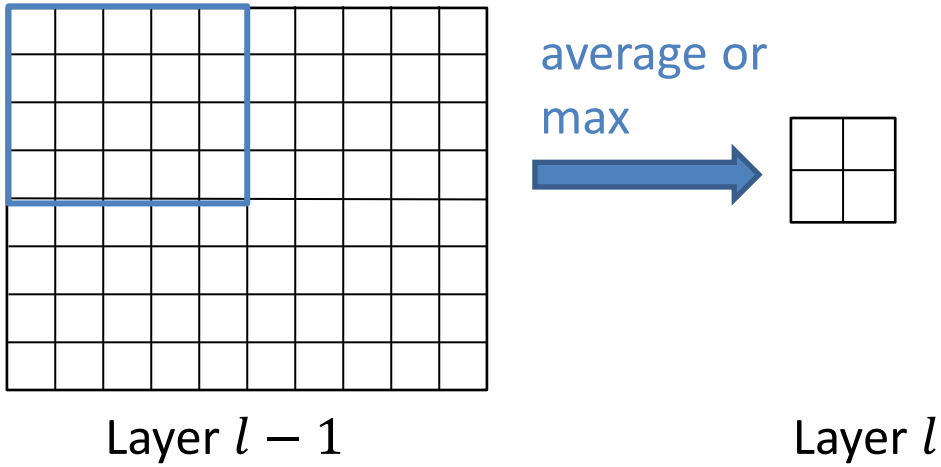
Translation invariance!

How do we model complex cells?



Pooling in local regions

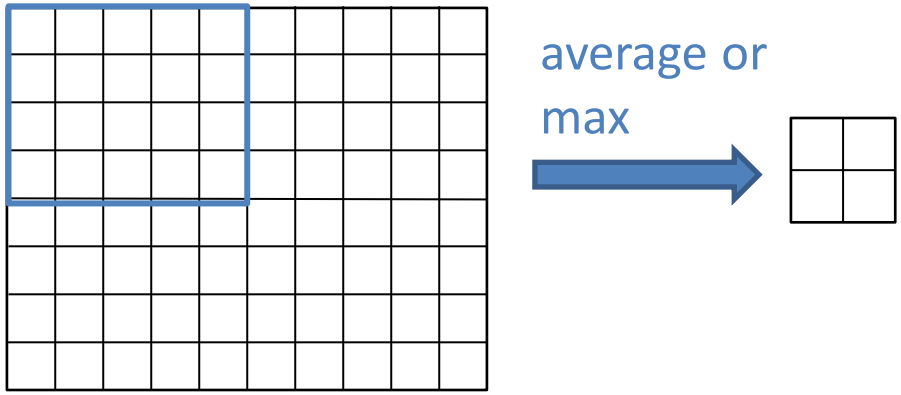
Poolingsize =4x5



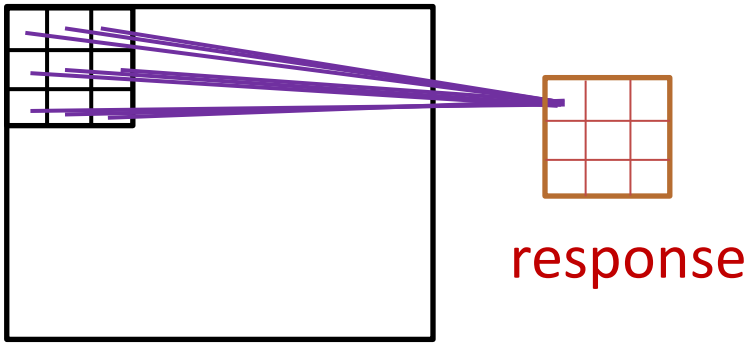
$$\boldsymbol{y}^{(l)} = \text{pooling}(\boldsymbol{y}^{(l-1)})$$

- Divide the convolved features into *disjoint* $m \times n$ regions, and take the mean (or maximum) feature activation over these regions to obtain the pooled features
- Similar operations on 1D input

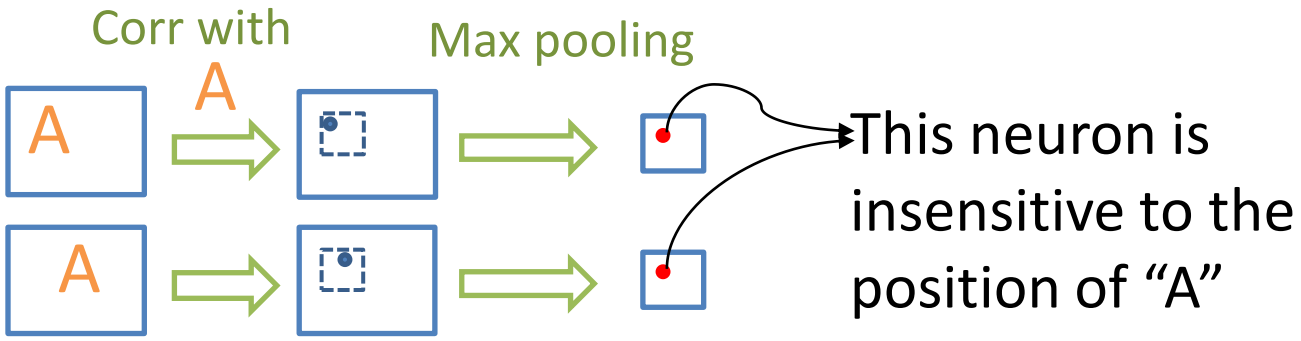
Interpretation from neural network perspective



From the perspective of neural network



Can pooling model the function of complex cells?



Simple cells: feature detector → Convolution

complex cells: translation invariant → Pooling

- Other advantages of max pooling?

Other advantages

- Reduce the number of features for final classification
  - Consider images of 96×96 pixels. Suppose we have learned 400 features over 8×8 inputs. This results in an output of size $(96 - 8 + 1)^2 \times 400 = 3{,}168{,}400$ features per example
- Enlarge the effective region of features in the next layer
  - A feature learned in the pooled maps will have larger effective regions in the pixel space

This is similar to the receptive fields of visual neurons, whose sizes increase along the visual hierarchy

- What if the pooling stride ≠ poolingsize

Overlapped pooling

- How about 3D input?

Channel-wise pooling

Construction of CNN

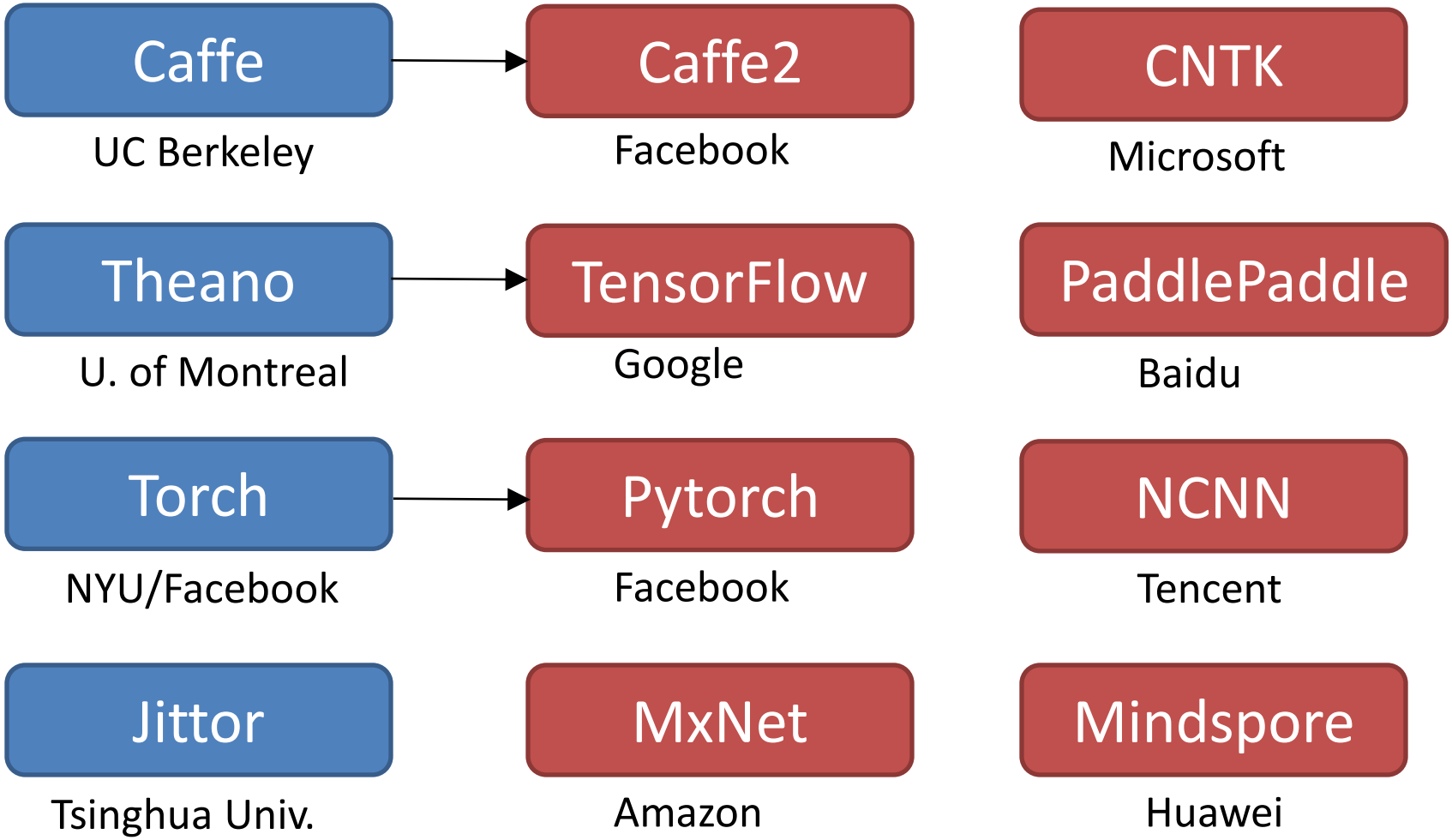
- The convolutional layers and pooling layers can be combined freely with other layers that we have discussed
  - Fully connected layer
  - Sigmoid layer, ReLU layer or other activation layers
  - Euclidean loss layer
  - Cross-entropy loss layer

  as well as other layers that we haven't discussed, e.g.,
  - Local response normalization layer (Krizhevsky et al. 2012)
  - Dropout layer (Srivastava et al., 2014)
  - Batch normalization layer (Ioffe and Szegedy, 2015)

CNN Implementation

- Implement each *type* of layer as a class and provide functions for forward calculation and backward calculation, respectively
- Design a CNN structure by specifying layer modules in a main file
- Run forward process
  - Calculate the output $\boldsymbol{y}^{(l)}$ for $l = 1,2, \dots, L$
- Run backward process
  - Calculate $\partial E/\partial \boldsymbol{W}^{(l)}$ and $\partial E/\partial \boldsymbol{b}^{(l)}$ if any, and $\boldsymbol{\delta}^{(l)}$ for $l = L, L-1, \dots, 1$
- Update $\boldsymbol{W}^{(l)}$ and $\boldsymbol{b}^{(l)}$ for $l = 1,2, \dots, L$

Frameworks


Caffe
UC Berkeley
Caffe2
Facebook
CNTK
Microsoft
Theano
U. of Montreal
TensorFlow
Google
PaddlePaddle
Baidu
Torch
NYU/Facebook
Pytorch
Facebook
NCNN
Tencent
Jittor
Tsinghua Univ.
MxNet
Amazon
Mindspore
Huawei

Demo: MNIST classification

https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html

MNIST

- 60,000 training images and 10,000 test images
- 28x28 black and white images



Network setting

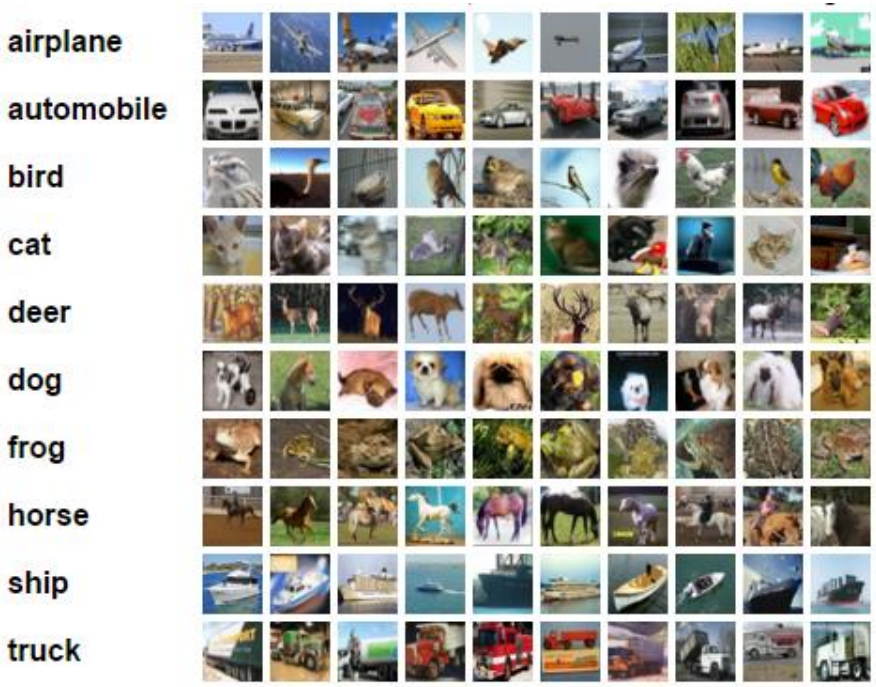
```
layer_defs = [];
layer_defs.push({type:'input', out_sx:24,
out_sy:24, out_depth:1});
layer_defs.push({type:'conv', sx:5, filters:8,
stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:2, stride:2});
layer_defs.push({type:'conv', sx:5, filters:16,
stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:3, stride:3});
layer_defs.push({type:'softmax',
num_classes:10});
```

Demo: CIFAR-10 classification

https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html

CIFAR-10 & CIFAR100

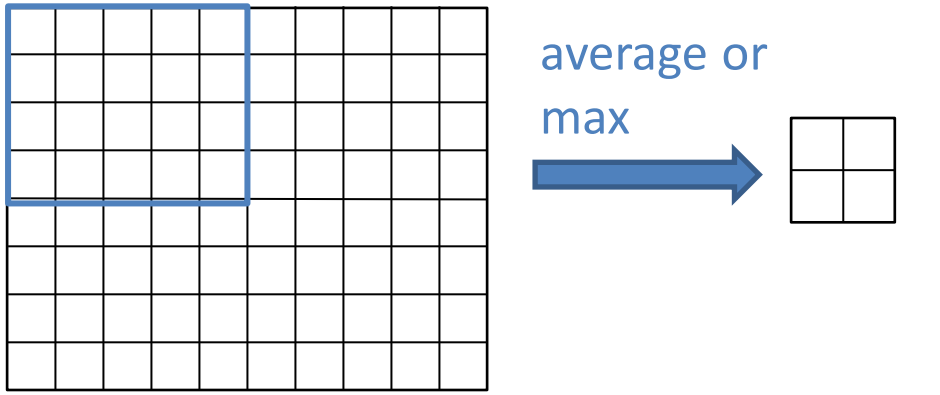
- 50,000 training images and 10,000 test images
- 32x32 colour images




```
layer_defs = [];
layer_defs.push({type:'input', out_sx:32,
out_sy:32, out_depth:3});
layer_defs.push({type:'conv', sx:5,
filters:16, stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:2,
stride:2});
layer_defs.push({type:'conv', sx:5,
filters:20, stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:2,
stride:2});
layer_defs.push({type:'conv', sx:5,
filters:20, stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:2,
stride:2});
layer_defs.push({type:'softmax',
num_classes:10});
```

Summary of Part 3

Pooling



CNN=MLP+Conv+Pooling

多选题 1分

设置

About the functions of pooling in a CNN for classification, which are correct?

A Increase the number parameters of the model

B Enlarge the RF of neurons in the ascending pathway

C Realize invariance with the aid of convolutional layers

D Reduce the number of features for the classifier

提交

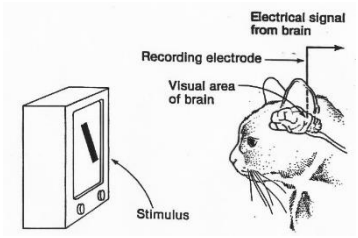
Outline

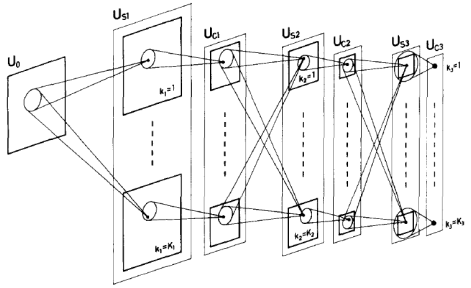
I. Introduction
II. Convolutional layer
III. Pooling layer
IV. Summary

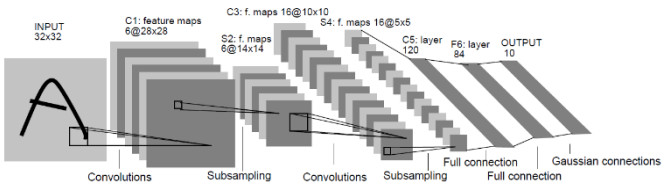
Summary of this lecture

knowledge

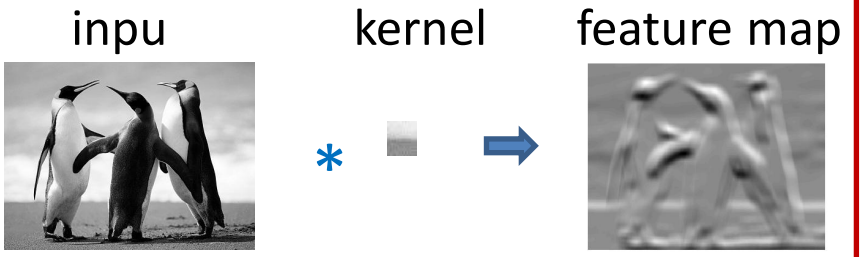
I. Introduction







II. Convolutional layer



III. Pooling layer



Summary of this lecture

Capability and value

- Neuroscience played a significant role in CNN, and should continue to play a significant role
- The ability to extract general principle from neuroscience findings and apply to a computational model

